



PermitParser

Minh Huynh, Omar Elmassaoudi, Asif Ali Khan, Melissa Rahman,
Younes Bouhaba, Alexander El Ghaoui, David Onwionoko, Steven
Zrihen, Youssef Yacoub, Hudson Lu, Allaye Dicko, Adib Akkari



Description of problem and domain

- Problem: Public building permit data across Greater Montreal is fragmented and unstructured, making it unusable for market analysis.
- Solution: PermitParser is an automated pipeline using Python and Docker on a serverless Google Cloud architecture to scrape and parse this data.
- Value: The system delivers a single, clean, analysis-ready dataset to our stakeholder to power their construction prospecting systems.



Major features

- **Scraper**
 - Automated discovery of permit documents (HTML, PDF).
 - Extracts from static sites, JS-portals, and document repositories.
 - Robust document upload to Google Cloud Storage.
- **Parser**
 - Extracts data from unstructured documents.
 - Optical Character Recognition (OCR) for scanned PDFs.
 - Standardizes data into a unified database schema.
- **Web Portal**
 - Project Management UI.
 - User Authentication (CRUD).
 - Project Configuration (CRUD).
 - Contact Form.

Product Demo - Scraper

```
src/services/scraper_service  dev python3.11 -m scrapers --municipality laval --scraper council --output FILE
Running laval/council scraper...
```

```
2025-11-12 13:57:26,854 [INFO] root: Starting Web Handler...
2025-11-12 13:57:27,120 [INFO] root: Navigating to: https://www.laval.ca/vie-democratique/hotel-de-ville-personnes-elues/ordre-jour-proces-verbaux-sommaire/
2025-11-12 13:57:32,305 [INFO] root: Page 1: Found 25 rows
2025-11-12 13:57:32,329 [INFO] root:   └─ Row 1: 12/11/2025 - ...
2025-11-12 13:57:32,340 [INFO] root:   └─ Row 2: 10/11/2025 - ...
2025-11-12 13:57:32,352 [INFO] root:   └─ Row 3: 29/10/2025 - ...
2025-11-12 13:57:32,361 [INFO] root:   └─ Row 4: 29/10/2025 - ...
2025-11-12 13:57:32,373 [INFO] root:   └─ Row 5: 29/10/2025 - ...
2025-11-12 13:57:32,384 [INFO] root:   └─ Row 6: 22/10/2025 - ...
2025-11-12 13:57:32,394 [INFO] root:   └─ Row 7: 22/10/2025 - ...
2025-11-12 13:57:32,404 [INFO] root:   └─ Row 8: 22/10/2025 - ...
2025-11-12 13:57:32,413 [INFO] root:   └─ Row 9: 15/10/2025 - ...
2025-11-12 13:57:32,423 [INFO] root:   └─ Row 10: 15/10/2025 - ...
2025-11-12 13:57:32,434 [INFO] root:   └─ Row 11: 15/10/2025 - ...
2025-11-12 13:57:32,447 [INFO] root:   └─ Row 12: 08/10/2025 - ...
2025-11-12 13:57:32,458 [INFO] root:   └─ Row 13: 08/10/2025 - ...
2025-11-12 13:57:32,470 [INFO] root:   └─ Row 14: 08/10/2025 - ...
2025-11-12 13:57:32,482 [INFO] root:   └─ Row 15: 01/10/2025 - ...
2025-11-12 13:57:32,496 [INFO] root:   └─ Row 16: 01/10/2025 - ...
2025-11-12 13:57:32,506 [INFO] root:   └─ Row 17: 01/10/2025 - ...
2025-11-12 13:57:32,516 [INFO] root:   └─ Row 18: 01/10/2025 - ...
```

```
src/services/scraper_service  dev python3.11 -m scrapers --list
```

```
== Available Scrapers ==
```

```
- LAVAL
  └─ council
  └─ donnees_quebec

- MONTREAL
  └─ role_evaluation
```

```
src/services/scraper_service  dev python3.11 -m scrapers --help
usage: __main__.py [-h] [--list] [--municipality MUNICIPALITY] [--scraper SCRAPER] [--output {STDOUT,FILE,GCS}]
```

Master scraper entry point for all municipalities

options:

```
-h, --help            show this help message and exit
--list                List all available scrapers and exit
--municipality MUNICIPALITY
                        Municipality to scrape (available: laval, montreal)
--scraper SCRAPER      Specific scraper to run
--output {STDOUT,FILE,GCS}
                        Output destination (default: FILE)
```

Examples:

```
python3.11 -m scrapers --municipality laval --scraper council --output FILE
python3.11 -m scrapers --list
```

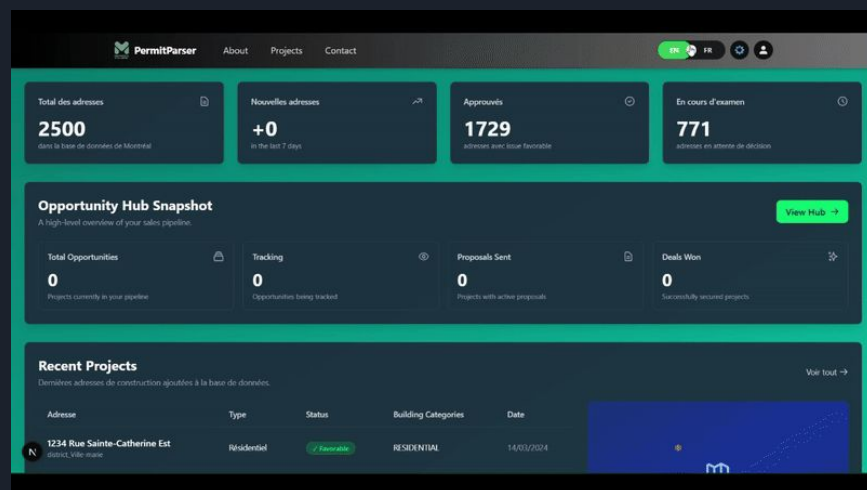
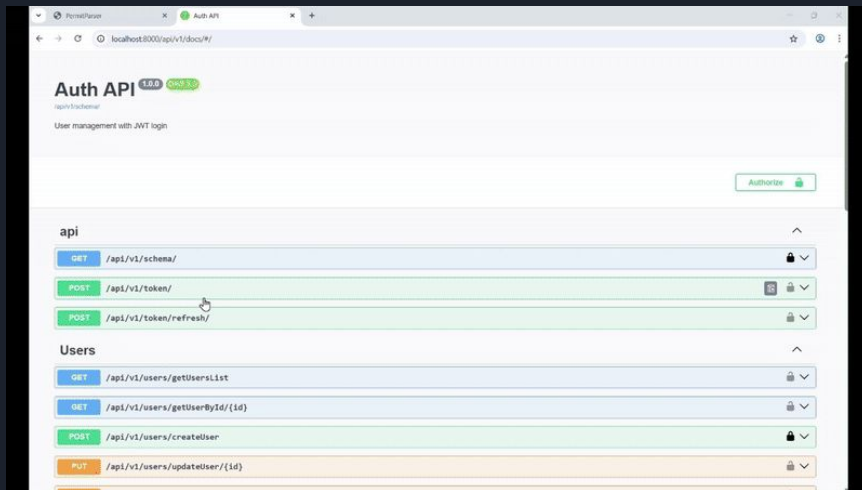
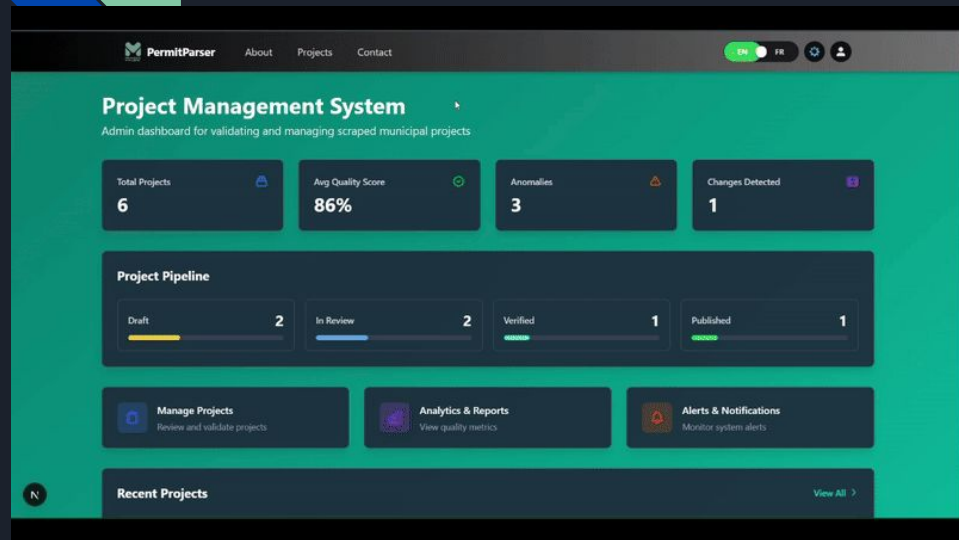
```
2025-11-12 14:00:13,587 [INFO] root: ✓ Fetched 1050 raw elements
2025-11-12 14:00:13,587 [INFO] root: Starting Parser...
2025-11-12 14:00:13,837 [INFO] root: Parser Statistics:
2025-11-12 14:00:13,837 [INFO] root:   └─ Total rows processed: 1050
2025-11-12 14:00:13,837 [INFO] root:   └─ Successfully parsed: 868
2025-11-12 14:00:13,837 [INFO] root:   └─ Skipped (invalid): 0
2025-11-12 14:00:13,837 [INFO] root:   └─ Skipped (duplicates): 182
2025-11-12 14:00:13,837 [INFO] root:   └─ Parse errors: 0
2025-11-12 14:00:13,837 [INFO] root: ✓ Parsed 868 unique records
2025-11-12 14:00:13,838 [INFO] root: Saving 868 documents to file...
2025-11-12 14:00:13,842 [INFO] lib.storage.local: ✓ Saved 868 documents to: data/laval/council/council_2025-11-12.jsonl
python3.11 -m scrapers --municipality laval --scraper council --output FILE 25.48s user 3.32s system 17% cpu 2:47.34 total
```



Product Demo - Parser

[Show video](#)

Demo - Web Portal



<https://github.com/munera-intelligence/PermitParser/wiki/Acceptance-Tests>



Deployment and User Feedback

- Discuss the level of deployment
 - Only test runs in google cloud (billing still at \$0)
- How did you get feedback from users?
 - N/A
- Discuss how feedback was incorporated into design and implementation
 - Feedback from stakeholder informed our architecture changes



Continued/Future rollout

- **Parser Refactoring:** Adapt parsing logic to align with the stakeholder's recently updated V2 data pipeline specification. Scraping components are unaffected.
- **Deployment Finalization:** Resolve outstanding Google Cloud IAM permissions to enable deployment of remaining automated workflows.
- **Web Portal Integration:** Define the final integration strategy and roadmap for the web portal based on ongoing stakeholder discussions.

<https://github.com/munera-intelligence/PermitParser/wiki/Deployment-Plan>



Economics evaluation of projects

- **Monetization is the stakeholder's responsibility.**
- This project's quantifiable outcome is a structured, analysis-ready dataset produced from raw public data.
- This data asset enables our stakeholder's downstream business intelligence and market prospecting systems.

<https://github.com/munera-intelligence/PermitParser/wiki/Economic-Impact>



Societal risks and mitigations

- **Information Asymmetry:** Creates a significant competitive advantage by converting inaccessible public data into proprietary market intelligence.
- **Accelerated Speculation:** Enables data-driven real estate speculation, which can accelerate gentrification and property value inflation.
- **Flawed Decision-Making:** Inherent data inaccuracies risk informing flawed, high-cost investment and urban planning decisions.

These risks are the direct consequence of Munera Intelligence's business model and cannot be fully mitigated beyond informing relevant stakeholders and ensuring clear data lineage.

<https://github.com/munera-intelligence/PermitParser/wiki/Risks-and-Mitigation>



Equity and Diversity

- DEI statement was moved to our wiki:
<https://github.com/munera-intelligence/PermitParser/wiki/Diversity-Statement>
- **Primary Equity Issue: Selection Bias**
 - Stakeholder priorities dictate which municipalities are parsed. This creates an information imbalance leaving communities unrepresented.
- **Our Commitment: Equitable Data Processing**
 - While we do not control selection, we are responsible for the equitable treatment of assigned data.
 - We apply uniform, rigorous processing and validation standards to all selected data sources, regardless of their municipal origin.



Planning and cost

- If you were consultants or employees, how much would the project cost?
 - Team leads have consistently quoted an estimate of \$5000-\$7000 per iteration
- Did you meet your iteration deadlines? Describe how
 - Due to changing requirements and permission bottlenecks, we have not
- Discuss any infrastructure or other costs
 - GitHub premium: \$56 USD monthly
 - GCP: none yet (free plan)

<https://github.com/munera-intelligence/PermitParser/wiki/Budget>

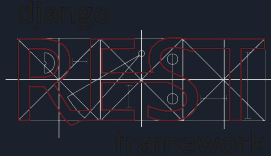


Architecture and domain diagrams

- Model: A Hybrid Orchestration-Choreography architecture.
 - Orchestration (Cloud Tasks): Used for transactional, reliable sequential steps (decomposition, aggregation).
 - Choreography (Pub/Sub): Used for scalable, high-performance parallel processing (parsing).
- Key Diagrams:
 - Component Diagram
 - Sequence Diagram
- Evolution & Patterns:
 - The architecture continuously adapted to the stakeholder's evolving pipeline.
 - A simple Strategy pattern was insufficient, leading to the natural adoption of a Factory Pattern to manage the growing complexity of unique, municipality-specific logic.

<https://github.com/munera-intelligence/PermitParser/wiki/System-Architecture>

Justification for frameworks and libraries



- Which frameworks and libs did you use and why?

- Web Portal : Django REST framework, [Next.js](#)

- Consistent integration with the full-stack website of the stakeholder

- Application: Google Cloud, Scrapy, Langchain

- Mandated by stakeholder

- If you were to do it again, would you use a different library?

- No. The core architectural choices and technology stack (Python, GCP, Docker) were dictated by stakeholder requirements



<https://github.com/munera-intelligence/PermitParser/wiki/Infrastructure-and-Tools>

Engineering tools

IDE: Visual Studio Code

Code Quality and Security: SonarQube

Coverage Testing: CodeCov

Storage and DB: Google Cloud Platform

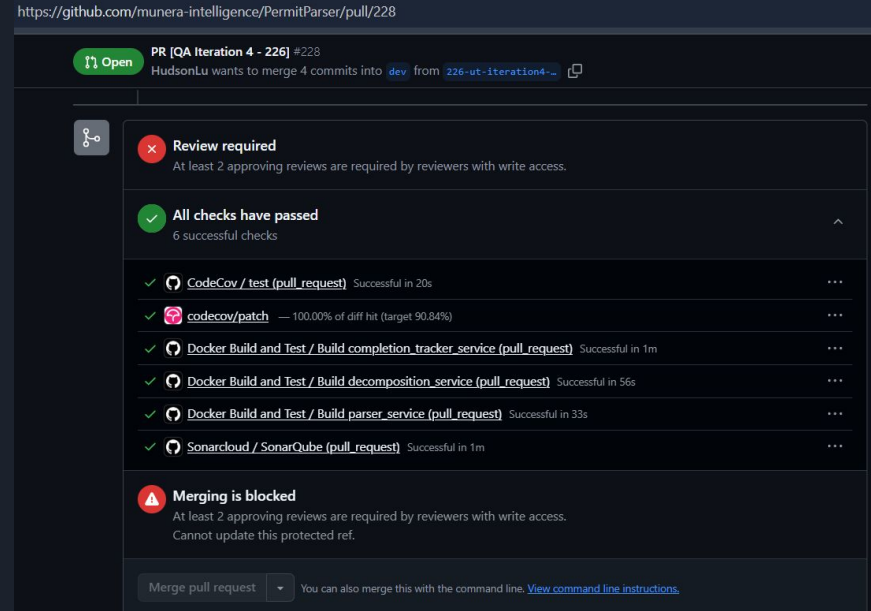
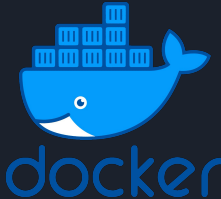
Containerization: Docker

<https://github.com/munera-intelligence/PermitParser/wiki/Infrastructure-and-Tools>



Google Cloud Platform

sonarqube



Testing and validation

Testing strategy

- 80% code coverage for unit tests (AAA) and testing business logic of all major components
- Manual testing from QA (Swagger UI)
- CI/CD pipeline for automated workflows

Effectiveness:

Early bug detection, fully functional features, and maintainable and reliable codebase

<https://github.com/munera-intelligence/PermitParser/wiki/QA-Test-Results>

Tracked lines	Covered	Partial	Missed	Coverage %
783	715	13	55	91.32%

munera-intelligence / PermitParser					
Coverage					
Source: latest commit: 5780b0					
YAML Configuration					
Learn more (2) about PR comments, targets, and badges					
Code tree					
File list					
20 total files: PermitParser					
Search for files					
Files	Tracked lines	Covered	Partial	Missed	Coverage %
PermitParser-Application/src/services/scrapper_service/its/storage/gcs.py	155	125	0	10	80.65%
PermitParser-Application/tests/unit/scrapper/test_gcs.py	108	108	0	0	100.00%
PermitParser-Application/tests/unit/scrapper/test_http.py	75	75	0	0	100.00%
PermitParser-Application/tests/unit/scrapper/test_storage_local.py	50	50	0	0	100.00%
PermitParser-Application/src/services/scrapper_service/its/storage/local.py	44	42	0	2	95.45%
PermitParser-Application/src/services/scrapper_service/its/http.py	44	41	1	2	93.18%
PermitParser-Application/tests/unit/test_us_2_1_login_session.py	32	32	0	0	100.00%
PermitParser-Application/tests/unit/scrapper/test_logging_conf.py	28	28	0	0	100.00%
PermitParser-Application/tests/unit/scrapper/test_cli.py	24	24	0	0	100.00%
PermitParser-Application/tests/unit/scrapper/test_config.py	21	21	0	0	100.00%
PermitParser-Application/tests/unit/test_us_2_2_static_html_listing.py	18	18	0	0	100.00%
PermitParser-Application/tests/unit/test_us_2_3_js_rendered_listing.py	18	18	0	0	100.00%
PermitParser-Application/tests/unit/test_us_2_5_navigation_catalog_resolver.py	18	18	0	0	100.00%
PermitParser-Application/src/services/scrapper_service/its/config.py	16	16	0	0	100.00%

```
(munera)
notAdmin 022:31 PermitParser/PermitParser-Application 227-task-reorganize-directory-structure + pytest tests/ --cov=src --cov-report=html

test session starts
platform darwin -- Python 3.14.0, pytest-9.0.0, pluggy-1.6.0
rootdir: /Users/notAdmin/Dev/PermitParser/PermitParser-Application
configfile: pytest.ini
plugins: anyio-4.11.0, cov-7.0.0
collected 51 items

tests/test_app.py .. [ 3%]
tests/test_us_6_2_5_workunit_to_pubsub_delivery.py s [ 5%]
tests/unit/scrapper/test_abstract_scraper.py .. [ 9%]
tests/unit/scrapper/test_cli.py .. [ 15%]
tests/unit/scrapper/test_config.py ... [ 21%]
tests/unit/scrapper/test_gcs.py ..... [ 35%]
tests/unit/scrapper/test_http.py ..... [ 49%]
tests/unit/scrapper/test_logging_conf.py ... [ 54%]
tests/unit/scrapper/test_storage_local.py ..... [ 68%]
tests/unit/test_us_1_1_pre_permit_laval_sources.py x. [ 72%]
tests/unit/test_us_2_1_login_session.py ... [ 78%]
tests/unit/test_us_2_2_static_html_listing.py ... [ 84%]
tests/unit/test_us_2_3_js_rendered_listing.py ... [ 90%]
tests/unit/test_us_2_4_api_based_source.py .. [ 94%]
tests/unit/test_us_2_5_navigation_catalog_resolver.py ... [100%]

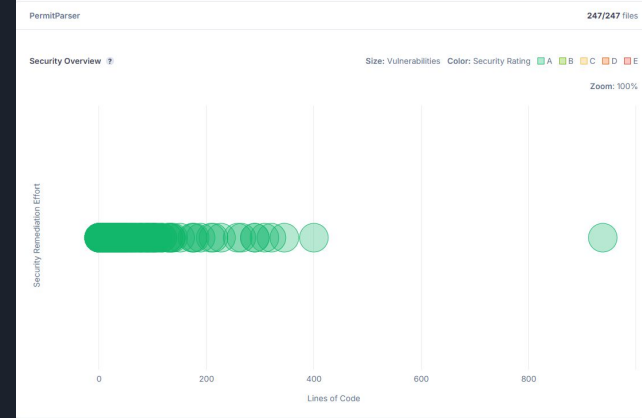
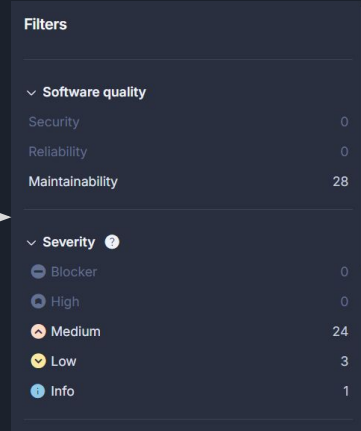
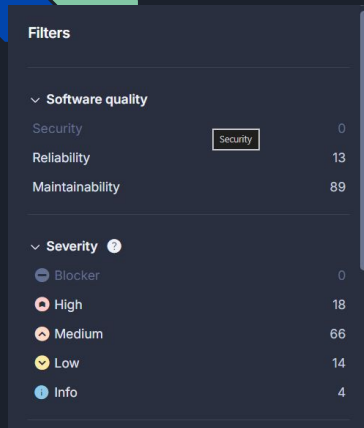
===== tests coverage =====
coverage: platform darwin, python 3.14.0-final-0

Coverage HTML written to dir htmlcov

===== 49 passed, 1 skipped, 1 xfailed in 3.91s =====

(munera)
```


Code Quality and Security



Last analysis: 12/11/2025, 00:28 • 8.8k Lines of Code • Python, TypeScript, ...



A 78
Maintainability

0.3%
Duplications

A 0
Security



Readme and other documentation

- Show us your readme and ensure it would allow any engineer to start working on the project: [README.md](#)
- Did your stakeholder require any other type of documentation
 - No, the wiki is sufficient
- Discuss your release notes
 - Laval scraping went well, moved to experimental scrapers
 - Parsing was bottlenecked by stakeholder and needs refactoring for Iteration 2
 - Expanded scope by creating improved web portal

<https://github.com/munera-intelligence/PermitParser/releases/tag/v1.0.0-alpha>



Code walkthrough

- Using the top 5 most substantial files and tests
- Do a code walkthrough discussing the **substantial engineering work done**
- You have 5 minutes for this.



List the Top 5 **most** important files

File path with clickable GitHub link	Purpose (1 line description)
Scraping GCS CRUD	Handles uploading PDFs and other files to Google Cloud Storage with retries, resumable uploads, gzip compression, and metadata.
Scraping Http Handler	Encapsulates outbound HTTP requests with per-host rate limiting, retries, and proper error handling.
Parsing Decomposer	Breaks down raw documents into structured work units (TEXT/TABLE/IMAGE) and publishes them to Pub/Sub for parallel processing.
Parsing Structure and saving	Extracts structured data from work units using BigQuery-aligned extraction functions and saves parsed objects to GCS for downstream analysis.
Web Portal Project API	Implements the core business logic of project management system, providing full CRUD operations on projects, project-status workflows, validation rules, and Swagger/OpenAPI documentation for the backend of the web portal



List the Top 5 most important **Tests**

Test file path with clickable GitHub link	Purpose (1 line description)
Scraping Test GCS	Comprehensive tests for file uploads, retries, gzip handling, PDF-specific naming, and metadata, using mocks and tmp files for isolation.
Scraping Test Http	Focused tests for rate-limiting, headers, retries, 404 handling, and network errors, with mocks and monkey patching for deterministic behavior.
Scraping Test Locale	Ensure storage module correctly saves documents in JSON/JSONL formats, handles errors properly, and generates accurate SHA-256 hashes for both bytes and files
Config Test Logging	Ensure logging setup works properly by setting the right log levels, adding color-formatted handlers, and preventing duplicate logger configurations.
Parser Test CLI	Ensure CLI parser works correctly by validating default arguments, allowed output options, and proper error handling for invalid choices.